

TIADC 校准技术深度分析

——从 Verilog ASIC 到 SoC C 语言实现

综合整理自 MATLAB 行为模型、Verilog 设计文档与 SoC 可行性分析

2026 年 5 月 18 日

摘要

本报告基于 64 通道 64 GSa/s 时间交织 ADC (TIADC) 校准系统的原始技术文档与 MATLAB 行为模型，系统梳理了 Offset / Gain / Timing Skew 三类校准的数学原理与实现细节。在此基础上，深入分析了在 SoC (FPGA + ARM) 平台上使用 C 语言替代纯 ASIC Verilog 方案进行校准的可行性，包括计算复杂度评估、实时性约束、定点化策略，并给出了完整的 C 语言参考实现与推荐架构方案。

目录

1	TIADC 校准技术：从 ASIC 到 SoC 的迁移分析	3
1.1	绪论：时间交织 ADC 的前世今生	3
1.1.1	为什么需要 TIADC?	3
1.1.2	本报告关注的 TIADC 系统	3
1.1.3	失配问题——理想与现实的差距	3
1.2	TIADC 的三种失配误差：从理论到现象	4
1.2.1	失调失配 (Offset Mismatch)	4
1.2.2	增益失配 (Gain Mismatch)	4
1.2.3	时钟偏斜失配 (Timing Skew Mismatch)	4
1.2.4	三种失配对比总结	5
1.3	ADC 行为模型：校准算法的试验场	5
1.3.1	非二进制（冗余）SAR ADC 模型——ADCF.m	5
1.3.2	二进制 SAR ADC 模型——ADCF_2.m	5
1.3.3	模型对比	5
1.4	第一步：Offset 校准原理与实现	6
1.4.1	核心原理	6
1.4.2	MATLAB 实现核心代码	6
1.4.3	收敛判断	6
1.4.4	溢出保护	7
1.5	第二步：Gain 校准原理与实现	7
1.5.1	核心原理	7

1.5.2	LMS 迭代算法	7
1.5.3	参考值选取的两种模式	7
1.5.4	MATLAB 实现核心代码	7
1.6	第三步: Timing Skew 校准原理与实现	8
1.6.1	核心原理——基于自相关算法	8
1.6.2	从 2 路扩展到 64 路	8
1.6.3	MATLAB 实现核心代码	8
1.6.4	多种实现变体	9
1.7	综合校准: 三管齐下	9
1.7.1	all_cali.m ——完整的校准流水线	9
1.7.2	校准效果验证	10
1.8	性能评估体系: 如何衡量校准好坏	10
1.8.1	ADC_dynamic_with_figure.m ——动态性能测试	10
1.8.2	关键指标定义	10
1.9	SoC C 语言实现可行性分析	11
1.9.1	问题定义: 从 ASIC 到 SoC 的范式转换	11
1.9.2	关键洞察: 校准参数估计 $\neq$ 校准参数应用	11
1.9.3	计算复杂度分析	12
1.9.4	实时性约束与带宽分析	13
1.9.5	推荐的 SoC 架构方案	14
1.9.6	C 语言实现的定点化策略	14
1.9.7	PS 端控制逻辑 C 实现	18
1.9.8	SoC 与纯 ASIC 方案对比	19
1.9.9	关键结论与建议	19
1.10	总结与展望	21
1.10.1	项目整体架构回顾	21
1.10.2	关键技术总结	21
1.10.3	改进方向	21
1.10.4	结语	21

1 TIADC 校准技术：从 ASIC 到 SoC 的迁移分析

1.1 绪论：时间交织 ADC 的前世今生

1.1.1 为什么需要 TIADC?

在现代电子系统中，模数转换器（ADC）是连接模拟世界与数字世界的桥梁。随着 5G/6G 通信、高速示波器、雷达系统和电子战技术的发展，对 ADC 采样率的要求已从 GSa/s（千兆采样/秒）迈向数十 GSa/s。

单个 ADC 的采样率受限于：

- **工艺限制**: CMOS 工艺的晶体管特征频率有限
- **功耗限制**: 高速 ADC 的功耗随采样率超线性增长
- **精度限制**: 高精度（高位宽）与高速度之间存在根本性矛盾

时间交织 ADC (TIADC) 应运而生——通过 M 个低速率子 ADC 并行工作，在不牺牲精度的前提下成倍提升采样率。

1.1.2 本报告关注的 TIADC 系统

表 1: 两套 TIADC 系统参数对比

系统	通道数	采样率
16 通道系统	16 路	16 GSa/s
64 通道系统	64 路	64 GSa/s

核心设计思路：每个子 ADC 以 f_s/M 的速率采样，但采样时钟相位依次错开 $T = 1/f_s$ 。结果，整体采样率等效为 f_s ，是单个子 ADC 的 M 倍。

1.1.3 失配问题——理想与现实的差距

理论上， M 个完全一致的子 ADC 并行工作就能实现完美的 M 倍采样率提升。但现实中的芯片制造存在**工艺偏差 (Process Variation)**，导致各通道之间存在三种失配：

1. **失调失配 (Offset Mismatch)** ——各通道直流偏置不同
2. **增益失配 (Gain Mismatch)** ——各通道放大倍数不同
3. **时钟偏斜失配 (Timing Skew Mismatch)** ——各通道采样时刻偏离理想位置

如果不校准，TIADC 的动态性能甚至不如单个子 ADC。

一个形象的比喻：想象 M 个人排成一排传递水桶。如果每个人的身高（失调）、臂力（增益）和出手时机（时钟偏斜）都不一样，那么整个传递链条的效率会大打折扣。校准就是要让所有人的动作协调一致。

1.2 TIADC 的三种失配误差：从理论到现象

1.2.1 失调失配 (Offset Mismatch)

物理本质：每个子 ADC 内部都有输入偏置电压（由比较器失调、电荷注入等引起）。即使是完全相同的版图设计，制造完成后每个通道的偏置电压也会略有不同。

数学描述——第 k 通道的输出：

$$y_k[n] = x_k[n] + O_k \quad (1)$$

其中 O_k 为第 k 通道的失调误差。

频谱影响：失调失配在频域引入位于 f_s/M 及其整数倍处的杂散 (spur)：

$$f_{\text{spur}} = k \cdot \frac{f_s}{M}, \quad k = 0, 1, 2, \dots, M-1 \quad (2)$$

这些杂散与输入信号频率无关——它们是**固定图案噪声**。

1.2.2 增益失配 (Gain Mismatch)

物理本质：各通道 ADC 的参考电压、电容阵列匹配程度不同，导致对同一输入信号的增益不同。

数学描述——第 k 通道的输出：

$$y_k[n] = (1 + \Delta G_k) \cdot x_k[n] \quad (3)$$

频谱影响：同样在 f_s/M 整数倍处引入杂散，但**杂散幅度与输入信号幅度成正比**——输入信号越大，增益失配的影响越严重。

1.2.3 时钟偏斜失配 (Timing Skew Mismatch)

物理本质：时钟分配网络的不理想导致每个子 ADC 的采样时钟边沿相位不能精确地间隔 T_s 。

为什么时钟偏斜最棘手？时钟偏斜引入的误差幅度与输入信号频率成正比：

$$\epsilon_k \approx A \cdot 2\pi f_{in} \cdot \Delta t_k \cdot \cos(2\pi f_{in} t_k) \quad (4)$$

这意味着：

- 对于低频信号，误差很小，可以忽略
- 对于高频信号（接近奈奎斯特频率），微小的时钟偏差会导致巨大的采样误差
- 这是限制 TIADC 在高频应用中的**最主要瓶颈**

时钟偏移分为两类：

- **时钟延时 (Clock Delay)：**固定的相位偏移，**可以被校准** ✓
- **时钟抖动 (Clock Jitter)：**随机的相位噪声，**无法被校准** ×

表 2: 三种失配误差对比

特性	失调失配	增益失配	时钟偏斜失配
物理根源	偏置电压不匹配	参考电压/器件不匹配	时钟分配不匹配
数学形式	加法性误差 $+O_k$	乘法性误差 $\times G_k$	时间位移 Δt_k
杂散位置	f_s/M 整数倍	f_s/M 整数倍	f_s/M 整数倍
与输入频率关系	无关	正比于信号幅度	正比于信号频率
校准难度	★★ 中等	★★ 中等	★★★ 最难
校准顺序	第 1 步	第 2 步	第 3 步

1.2.4 三种失配对比总结

1.3 ADC 行为模型：校准算法的试验场

在实际芯片流片之前，需要用 MATLAB 行为模型来验证校准算法。原始项目提供了多种 ADC 行为模型：

1.3.1 非二进制（冗余）SAR ADC 模型——ADCF.m

传统二进制 SAR ADC 每一位的权重是 2 的幂次，而非二进制 ADC 采用**冗余权重**设计——权重之和大于满量程范围，给比较器判决留出了纠错空间。

权重序列为 $[60, 28, 16, 10, 6, 4, 2, 1, 0.5]$ ，对应输出码字权重为 $[120, 56, 32, 20, 12, 8, 4, 2, 1]$ 。

关键洞察：由于权重之和 $(127.5) >$ 满量程范围 (128) ，比较器在前面的判决即使出错，后续位还能纠回来。而二进制 ADC 一旦高位判决错误，误差无法挽回。

1.3.2 二进制 SAR ADC 模型——ADCF_2.m

标准二进制逐次逼近 ADC，从 MSB 到 LSB 依次判断。例如 8 位 ADC 的判断阈值依次为： $FS/2, FS/4, FS/8, FS/16, FS/32, FS/64, FS/128, FS/256$ 。

1.3.3 模型对比

表 3: ADC 行为模型对比

特性	ADCF.m (非二进制)	ADCF_2.m (二进制)
权重序列	$[60, 28, 16, 10, 6, 4, 2, 1, 0.5]$	标准的 2 的幂次
输出码字权重	$[120, 56, 32, 20, 12, 8, 4, 2, 1]$	标准的二进制
冗余度	有（权重和 $>$ 范围）	无
适用场景	非二进制 ADC 校准	二进制 ADC 校准

1.4 第一步：Offset 校准原理与实现

1.4.1 核心原理

基本思想：对正弦信号大量采样，理论上均值为 0。但由于存在失调偏移，均值不为 0——这个均值就是失调误差的估计值。

本项目采用**指数平均法**（一阶 IIR 低通滤波器）：

$$\bar{x}(n) = \bar{x}(n-1) + \alpha[x(n) - \bar{x}(n-1)] \quad (5)$$

具有以下优点：

- 输出连续，每次采样都更新
- 平滑度高，没有突变
- 硬件实现简单（只需加法器 + 移位寄存器，无需乘法器）

1.4.2 MATLAB 实现核心代码

Listing 1: Offset 校准核心迭代

```

1  for i = 1:j-1
2      for r = 1:M % M=16 通道
3          % 生成带失调误差的输入信号
4          Vi = 0.6*cos(2*pi*fin1*t) + offset_mismatch(r);
5
6          % ADC 量化
7          Ao(i,r) = ADCF(Vi) - 128;
8
9          % 指数平均迭代
10         int(r,i+1) = int(r,i) + w(1) * Ao(i,r);
11
12         % 截取整数部分（模拟硬件中的位宽限制）
13         int_limit(r,i+1) = floor(int(r,i+1) * 4) / 4;
14     end
15 end

```

1.4.3 收敛判断

通过监测 offset 估计值的变化方向来判断收敛：

- 统计增大和减小的次数
- 当两者相等时，认为已收敛

1.4.4 溢出保护

由于硬件寄存器位宽有限，offset 估计值可能溢出。在 ASIC 实现中通过符号位检测实现饱和处理。在 SoC 软件实现中可直接用条件判断，更灵活。

1.5 第二步：Gain 校准原理与实现

1.5.1 核心原理

增益校准的目标不是消除每个通道的绝对增益误差，而是**消除通道间的相对增益误差**——即让所有通道的增益相等。

引入增益补偿系数 $g'_1, g'_2, \dots, g'_M$ 后，需要满足：

$$1 + g_1 + g'_1 = 1 + g_2 + g'_2 = \dots = 1 + g_M + g'_M \quad (6)$$

1.5.2 LMS 迭代算法

通过比较各通道输出信号的**平均功率**与**参考功率** T_h 来调整增益补偿系数：

$$g'_i[n+1] = g'_i[n] + \alpha(T_h[n] - y_i[n]^2) \quad (7)$$

- 如果 $y_i[n]^2 > T_h \rightarrow$ 减小增益补偿系数
- 如果 $y_i[n]^2 < T_h \rightarrow$ 增大增益补偿系数
- 当各通道输出功率都收敛到 T_h 时，相对增益误差被消除

1.5.3 参考值选取的两种模式

表 4: 参考值选取模式

模式	描述	特点
模式 0	以通道 1 的瞬时功率为参考	简单，但通道 1 的误差会传播
模式 1	以所有通道瞬时功率均值为参考	更鲁棒，是默认选择

1.5.4 MATLAB 实现核心代码

Listing 2: Gain 校准核心迭代

```

1 beta = 1/2^26;           % 带宽控制
2 step_size = 0.05/31;     % 增益补偿系数步长（6位控制字）
3
4 for i = 1:j-1
5     for r = 1:M
6         % 应用增益补偿

```

```

7      Vi = (1+F4(r,i)*step_size) * gain_mismatch(r) * 0.4*cos(...) +
      0.45;
8      Ao(i,r) = ADCF_2(Vi, 8, 0.9, 0) * 256 - 128;
9
10     % 瞬时功率
11     F1(r,i+1) = Ao(i,r) * Ao(i,r);
12
13     % LMS 迭代
14     F3(r,i+1) = F3(r,i) - beta * (F1(r,i+1) - th_fl(1,i));
15
16     % 提取增益补偿系数（截取前9位）
17     F4(r,i+1) = floor(F3(r,i+1) * 512);
18 end
19 end

```

1.6 第三步：Timing Skew 校准原理与实现

1.6.1 核心原理——基于自相关算法

这是三种校准中最复杂、也最精妙的一种。

从 2 路交织开始理解：考虑 2 路交织 ADC，通道 1 为参考（理想），通道 2 存在时间偏斜 ΔT 。

关键观察：如果通道 2 的采样时刻偏左 ($\Delta T < 0$)，则 $y_1[k-1]$ 与 $y_2[k-1]$ 的相关性增强；如果偏右，则 $y_2[k-1]$ 与 $y_1[k]$ 的相关性增强。

数学表达——自相关差值：

$$D_{\Delta t} = R_x(T_{ck} - \Delta T) - R_x(T_{ck} + \Delta T) \quad (8)$$

利用自相关函数的偶函数特性，当 ΔT 足够小时，用泰勒展开：

$$D_{\Delta t} = -2\Delta T \left. \frac{dR_x}{d\tau} \right|_{\tau=T_{ck}} \quad (9)$$

结论： $D_{\Delta t}$ 的符号指示了偏移方向，大小指示了偏移程度。

1.6.2 从 2 路扩展到 64 路

在 64 通道系统中，通道被组织为 4 组，每组由同一个 16GHz 时钟驱动：

由于 4 组时钟共享同一时钟源，只需校准 3 个时钟偏斜参数 ($\Delta t_2, \Delta t_3, \Delta t_4$)，通道 1 的时钟组 ϕ_1 作为参考。

1.6.3 MATLAB 实现核心代码

Listing 3: Timing Skew 校准核心迭代

```

1 for i = 1:N/64-1
2     for t = 1:64

```


表 5: 64 通道时钟分组

时钟组	通道编号
ϕ_1 (参考)	1, 5, 9, 13, ..., 61
ϕ_2	2, 6, 10, 14, ..., 62
ϕ_3	3, 7, 11, 15, ..., 63
ϕ_4	4, 8, 12, 16, ..., 64

```

3      % 生成带时间偏斜的输入信号
4      if mod(t,4) == 0
5          Vi = cos(2*pi*f*(t + skew(4) - mismatch_1(i,4)*step_size));
6      elseif mod(t,4) == 1
7          Vi = cos(2*pi*f*(t + skew(1))); % 参考组
8      else
9          Vi = cos(2*pi*f*(t + skew(mod(t,4)) - mismatch_1(i,mod(t,4))*
              step_size));
10     end
11     Ao(i,t) = ADCF(Vi) - 128;
12 end
13
14 % 计算64个通道两两相邻的自相关之和
15 % 共16组：A1~A16
16 A17 = A1 + A2 + ... + A16; % 总自相关和
17
18 % 计算每个非参考通道的误差
19 for t = 2:4
20     % LMS迭代更新时间补偿参数
21     mismatch(i+1,t) = mismatch(i,t) + u * e(i,t);
22     mismatch_1(i+1,t) = floor(mismatch(i+1,t) / 2^24);
23 end
24 end

```

1.6.4 多种实现变体

1.7 综合校准：三管齐下

1.7.1 all_cal_i.m ——完整的校准流水线

all_cal_i.m 是项目中最综合的校准脚本，在一次仿真中同时实现三种校准。校准顺序至关重要，不可调换：

1. **Offset 校准**：如果存在直流偏移，后续的功率计算（用于增益校准）会引入偏差
2. **Gain 校准**：增益误差会影响信号幅度，进而影响自相关计算的准确性

表 6: Timing Skew 校准的实现变体

文件	通道数	ADC 类型	特点
time_loop.m	16	通用 ADC	步长极小 $u = 2^{-70}$ ，精度极高但收敛慢
time_not2.m	64	非二进制	仅含时间校准，4 组时钟
time_not2_1.m	64	二进制	二进制 ADC 版本，带直流偏置
time_onlyone.m	64	非二进制	简化为 8 通道相关
all_cali.m	64	两者皆支持	最完整版本 ，三重校准同时进行

3. Timing Skew 校准：时间偏斜不影响幅度和直流偏置，放在最后

1.7.2 校准效果验证

所有校准脚本都使用 FFT 频谱分析进行校准前后的对比。典型改善效果：

表 7: 校准前后性能对比（典型值）

指标	校准前	校准后	改善
SNR	~25 dB	~48 dB	+23 dB
SFDR	~35 dBc	~60 dBc	+25 dB
ENOB	~4 bits	~7.5 bits	+3.5 bits

1.8 性能评估体系：如何衡量校准好坏

1.8.1 ADC_dynamic_with_figure.m ——动态性能测试

这是项目中最重要评估函数，通过 FFT 分析计算 ADC 的关键性能指标。

步骤：去直流 → 加窗（可选）→ FFT → 搜索基频 → 搜索谐波（2~9 次）→ 计算指标 → 绘图。

1.8.2 关键指标定义

- **SNR（信噪比）**： $SNR = 10 \log_{10} \left(\frac{P_{signal}}{P_{noise}} \right)$ ，排除谐波分量
- **SINAD（信纳比）**： $SINAD = 10 \log_{10} \left(\frac{P_{signal}}{P_{noise} + P_{distortion}} \right)$ ，含噪声和谐波
- **ENOB（有效位数）**： $ENOB = \frac{SINAD - 1.76}{6.02}$ ，理想的 N 位 ADC 能达到 N 位 ENOB
- **SFDR（无杂散动态范围）**：信号与最大杂散之间的差距，是 TIADC 校准最直观的指标

- **THD (总谐波失真)** : $THD = 10 \log_{10} \left(\frac{\sum_{k=2}^9 P_{harmonic_k}}{P_{signal}} \right)$

1.9 SoC C 语言实现可行性分析

1.9.1 问题定义：从 ASIC 到 SoC 的范式转换

原始 TIADC 校准方案基于 Verilog 硬件描述语言，面向 ASIC（专用集成电路）实现。其核心设计理念是：

- 全硬件并行处理——每个通道拥有独立的校准运算单元
- 定点数运算——位宽严格受限（如 offset 29 位、gain 9 位控制字）
- 多档位 α 自动切换——通过硬件状态机实现
- 硬件溢出保护——通过门级逻辑实现饱和处理

本报告的目标是分析在 SoC（片上系统）平台上，使用 C 语言实现同样校准算法的可行性和约束条件。

1.9.2 关键洞察：校准参数估计 $\neq$ 校准参数应用

在讨论 C 语言可行性之前，必须先澄清一个极易混淆的概念——这也是 ASIC 方案和 C 方案之争的根节点：

表 8: 校准参数估计 vs 校准参数应用

维度	校准参数应用	校准参数估计
做什么	每采样点减去 offset、乘以 gain、调整时钟相位	从数据中估算 offset/gain/skew 的真实值
频率要求	必须每采样一次 (64 GHz)	可以大幅降采样 (kHz 级即可)
单次运算量	极轻 (1 次加减 +1 次移位/通道)	较重 (LMS 迭代、自相关、FFT)
适合什么	FPGA 硬连线 (PL)	C 代码在 CPU 上跑 (PS)
核心原因	确定性强、延迟低	灵活性强、逻辑复杂

核心结论：失配参数 (offset、gain、timing skew) 的变化源是温度漂移和器件老化，时间常数是秒到分钟级别。完全不需要每 15.6 ps 就重新估计一次——每毫秒估计一次都绰绰有余。

为什么校准参数估计可以大幅降采样？ 以三种校准分别分析：

Offset 校准 (指数平均法) : $\bar{x}(n) = \bar{x}(n-1) + \alpha[x(n) - \bar{x}(n-1)]$ 本质是一阶 IIR 低通滤波器，提取 DC 分量。即使每 1024 个采样才喂入一个点，只要适当调整 α ，最终收敛值完全一样。

Gain 校准 (LMS 功率均衡)：需要的是长期平均功率而非瞬时功率。可以用 C 代码累加 1024 个 y_i^2 值，算平均后做一次 LMS 更新，更新频率降到 ~ 62.5 kHz。

Timing Skew 校准 (自相关法)：自相关的估计需要大量样本才准确——本来就该用大量采样点做统计。每 64×1024 个采样（约 $1 \mu\text{s}$ 的数据）做一次互相关计算，更新频率 ~ 1 MHz。

降采样后的 CPU 负载分析 假设降采样比为 1024:1，64 通道，ARM Cortex-A53 @1.2 GHz：

表 9: 降采样后 C 代码 CPU 负载估算

任务	有效输入率	运算量/次	CPU 负载
Offset IIR (64 通道)	62.5 kHz	~ 5 指令/通道	$< 0.02\%$
Gain LMS (64 通道)	62.5 kHz	~ 10 指令/通道	$< 0.04\%$
Timing 自相关	~ 1 MHz	~ 200 指令	$< 0.02\%$
FFT 监测 (16K 点)	10 Hz	$\sim 500\text{K}$ 指令	$< 0.5\%$
总计			$< 1\% \text{ CPU}$

结论：C 代码在 ARM Cortex-A53 上的 CPU 占用不到 1%。大部分时间 CPU 在休眠。

表 10: 不同场景对校准参数估计频率的需求

场景	需要高采样率估计?	原因
芯片出厂校准	× 不需要	一次性的，可用外部仪器辅助
上电启动校准	× 不需要	数据静止，毫秒级收敛即可
温度漂移跟踪	× 不需要	温度变化是秒级，kHz 远超需求
老化补偿	× 不需要	老化是月/年尺度
实时数据补偿	✓ 必须	每个采样点都要修正，但这是「应用」不是「估计」

高采样率校准是否必须？

推荐数据流架构 这就是 PL + PS 混合架构的核心价值：把高频简单操作留给 FPGA（每采样应用校准参数），把低频复杂决策留给 C 代码（降采样后估计校准参数）。两者解耦后，C 代码的实时性要求从 $15.6 \mu\text{s}$ 骤降到 $\sim 16 \mu\text{s}$ （降采样 1024 倍），变得完全可行。

1.9.3 计算复杂度分析

Offset 校准 (每个通道、每次采样)

Gain 校准 (每个通道、每次采样)

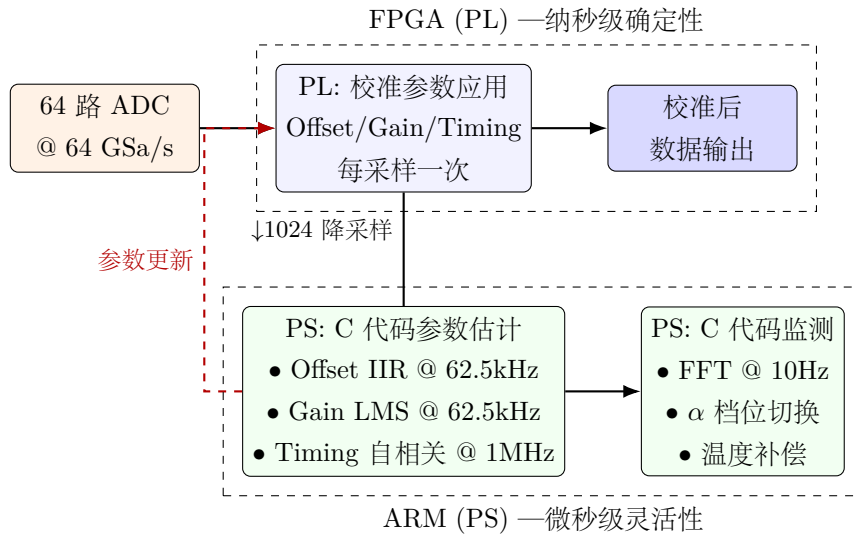


图 1: 推荐数据流架构：参数应用和参数估计完全解耦

表 11: Offset 校准单次迭代运算量

运算	操作	说明
指数平均	1 次乘法 + 1 次加法	$I = I + w \cdot A_o$
截位量化	1 次乘/移位 + 1 次截断	$\hat{O} = \lfloor I \times 4 \rfloor / 4$
减法补偿	1 次减法	$y = A_o - \hat{O}$
合计/通道/采样	2 次乘法 + 2 次加法	

Timing Skew 校准（每 64 次采样更新一次）

总运算量汇总

1.9.4 实时性约束与带宽分析

时间预算 对于 64 GSa/s TIADC 系统，每个采样的时间预算为：

$$T_s = \frac{1}{64 \times 10^9} \approx 15.6 \text{ ps} \quad (10)$$

这是一个极为苛刻的时间约束。

对于 16 GSa/s 系统，时间预算相对宽松：

$$T_s = \frac{1}{16 \times 10^9} \approx 62.5 \text{ ps} \quad (11)$$

C 语言在 SoC 上的实时性分析 现代 SoC（如 Zynq UltraScale+、Kria K26 等）的 ARM Cortex-A53 主频约 1.2–1.5 GHz，单周期约 0.7–0.8 ns。在如此紧张的时隙内，纯软件方案面临巨大挑战：

表 12: Gain 校准单次迭代运算量

运算	操作	说明
功率计算	1 次乘法	$P = A_o^2$
LMS 更新	1 次减法 + 1 次乘法 + 1 次加法	$F_3 = F_3 - \beta \cdot (P - \bar{P})$
控制字提取	1 次乘法/移位 + 1 次截断	$F_4 = \lfloor F_3 \times 512 \rfloor$
增益补偿	1 次乘法 + 1 次加法	$y = (1 + F_4 \cdot \Delta) \cdot x$
合计/通道/采样	4 次乘法 + 3 次加法	

表 13: Timing Skew 校准每次更新运算量

运算	操作	说明
64 通道互相关	64 次乘法 + 64 次加法	所有相邻通道对
16 组累加	48 次加法	$A_1 \sim A_{16}$
全局平均	15 次加法 + 1 次移位	$\bar{A} = \sum A_m / 16$
误差计算 (×3 组)	6 次乘法 + 6 次加法	每组需乘法和加法
LMS 更新 (×3 组)	3 次乘法 + 3 次加法	$\Delta t = \Delta t + u \cdot e$
控制字截位	3 次移位 + 3 次截断	
合计/64 采样	73 次乘法 + 139 次加法	折算: ~1.1 次乘法/采样

1.9.5 推荐的 SoC 架构方案

架构分工:

- **PL (Programmable Logic):** 用 Verilog/HLS 实现 Offset/Gain/Timing Skew 的实时校准运算数据通路。这些运算具有高度规则性和并行性，适合 FPGA 实现
- **PS (Processing System):** ARM 核运行 C 代码，负责：
 - α 档位切换策略（根据收敛状态动态调整步长）
 - 收敛检测（统计增大/减小次数）
 - 周期性的 FFT 性能监测
 - 温度补偿与参数持久化
 - 与上位机的通信（如通过 Ethernet/UART 上报校准状态）

1.9.6 C 语言实现的定点化策略

在 SoC 的 PS 端或纯粹嵌入式 MCU 上实现校准逻辑时，定点化是关键。下面给出 Offset 校准的 C 语言参考实现。

表 14: 64 通道系统总运算量（每采样周期）

校准类型	乘法/采样	加法/采样	更新频率
Offset	$64 \times 2 = 128$	$64 \times 2 = 128$	每采样
Gain	$64 \times 4 = 256$	$64 \times 3 = 192$	每采样
Timing	~ 1.1	~ 2.2	每 64 采样
总计	~ 385	~ 322	

表 15: 实时性方案对比

方案	每采样可用周期数	可行性
64 GSa/s + 纯 CPU	15.6ps/0.7ns ≈ 0.02 周期	完全不可行
16 GSa/s + 纯 CPU	62.5ps/0.7ns ≈ 0.09 周期	不可行
1 GSa/s + 纯 CPU	1ns/0.7ns ≈ 1.4 周期	勉强，需 SIMD 优化
推荐方案	FPGA 硬件加速 + CPU	可行

Listing 4: Offset 校准: C 语言定点实现

Offset 校准 C 实现

```

1  #include <stdint.h>
2
3  #define M_CHANNELS    64          // 通道数
4  #define ALPHA_SHIFT  12          // alpha = 2^{-12}
5  #define QUANT_SHIFT   2          // 截位: *4 >> 2
6
7  typedef struct {
8      int32_t accum;              // 29位累加器 (Q4.25)
9      int8_t  offset_est;         // 8位失调估计值
10 } offset_calib_t;
11
12 // 每个通道一个实例
13 offset_calib_t offset_ctx[M_CHANNELS];
14
15 // 每采样调用一次 (ISR 或 DMA 回调中)
16 static inline int16_t offset_calibrate(uint8_t ch, int16_t adc_raw)
17 {
18     offset_calib_t *ctx = &offset_ctx[ch];
19
20     // 指数平均: accum += alpha * adc_raw
21     // alpha = 2^{-12}, 用算术右移实现除法

```

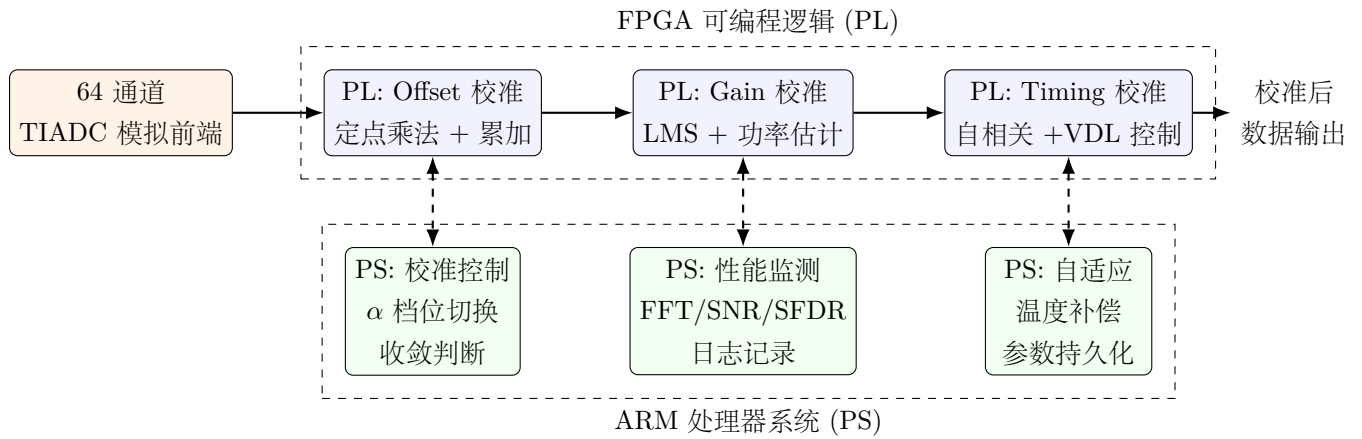


图 2: 推荐 SoC 架构: PL (FPGA) 负责实时校准运算, PS (ARM) 负责控制与监测

```

22     ctx->accum += (int32_t)adc_raw >> ALPHA_SHIFT;  // Q4.25
23
24     // 截位: 取 accum[28:21] 作为 8-bit offset
25     // 等效于 floor(accum * 4) / 4
26     ctx->offset_est = (int8_t)((ctx->accum >> 21) & 0xFF);
27
28     // 补偿输出
29     return adc_raw - ctx->offset_est;
30 }

```

Gain 校准 C 实现

Listing 5: Gain 校准: C 语言定点实现

```

1  #define GAIN_BETA_SHIFT  26          // beta = 2^{-26}
2  #define GAIN_CTRL_SHIFT  9          // 9位控制字
3  #define GAIN_STEP_Q      16          // step_size = 0.05/31, 用Q16定点
4
5  typedef struct {
6      int32_t lms_accum;              // LMS累加器
7      int16_t gain_ctrl;              // 9位增益控制字 (-256..255)
8  } gain_calib_t;
9
10 gain_calib_t gain_ctx[M_CHANNELS];
11
12 // 参考功率由PS端周期性更新
13 static int32_t ref_power = 0;
14
15 static inline int16_t gain_calibrate(uint8_t ch, int16_t adc_val)
16 {
17     gain_calib_t *ctx = &gain_ctx[ch];
18
19     // 瞬时功率

```



```

20     int32_t power = (int32_t)adc_val * adc_val;
21
22     // LMS: lms_accum -= beta * (power - ref_power)
23     int32_t delta = power - ref_power;           // 功率差
24     ctx->lms_accum -= delta >> GAIN_BETA_SHIFT;  // LMS更新
25
26     // 提取控制字
27     ctx->gain_ctrl = (int16_t)(ctx->lms_accum >> GAIN_CTRL_SHIFT);
28
29     // 增益补偿: y = (1 + gain_ctrl * step) * adc_val
30     int32_t gain = (1 << GAIN_STEP_Q) +
31                 (ctx->gain_ctrl * GAIN_STEP_CONST); // Q16
32     return (int16_t)((((int32_t)adc_val * gain) >> GAIN_STEP_Q));
33 }

```

Listing 6: Timing Skew 校准: C 语言实现

Timing Skew 校准 C 实现

```

1  #define N_GROUPS      4           // 时钟组数
2  #define CH_PER_GROUP  16          // 每组16通道 (64/4)
3  #define LMS_U_SHIFT   9           // u = 2^9 (注意是正幂!)
4  #define CTRL_SHIFT    24          // 控制字截位
5
6  typedef struct {
7      int64_t skew_accum;           // 时间偏斜累加器 (宽位宽)
8      int32_t skew_ctrl;           // 截位后的控制字
9  } timing_calib_t;
10
11 timing_calib_t timing_ctx[N_GROUPS]; // 组1为参考, 只校准2/3/4
12
13 // 存储64通道最新采样值
14 static int16_t sample_buf[64];
15
16 // 每64采样调用一次
17 void timing_skew_calibrate(void)
18 {
19     // 1. 计算16组互相关 (A1~A16)
20     int32_t A[16] = {0};
21     for (int m = 0; m < 16; m++) {
22         int base = m * 4;
23         // 组内4对: (0,1) (1,2) (2,3) (3,0)
24         A[m] = (int32_t)sample_buf[base+0] * sample_buf[base+1]
25             + (int32_t)sample_buf[base+1] * sample_buf[base+2]
26             + (int32_t)sample_buf[base+2] * sample_buf[base+3]
27             + (int32_t)sample_buf[base+3] * sample_buf[(base+4) % 64];
28     }

```

```

29
30 // 2. 全局平均
31 int32_t avg = 0;
32 for (int m = 0; m < 16; m++) avg += A[m];
33 avg >>= 4; // /16
34
35 // 3. 对每组(2,3,4)计算误差并LMS更新
36 for (int g = 1; g < 4; g++) { // g=1即phi2, g=2即phi3, g=3即phi4
37     // 计算该组在所有16个位置的互相关
38     int32_t corr_sum = 0;
39     for (int m = 0; m < 16; m++) {
40         int idx1 = g - 1 + m * 4; // 前一个通道
41         int idx2 = g + m * 4; // 后一个通道
42         corr_sum += (int32_t)sample_buf[idx1]
43                     * sample_buf[idx2 % 64];
44     }
45
46     // 误差信号
47     int32_t err = avg - corr_sum;
48
49     // LMS更新
50     timing_ctx[g].skew_accum += (int64_t)err << LMS_U_SHIFT;
51
52     // 控制字截位
53     timing_ctx[g].skew_ctrl =
54         (int32_t)(timing_ctx[g].skew_accum >> CTRL_SHIFT);
55 }
56 }

```

1.9.7 PS 端控制逻辑 C 实现

Listing 7: PS 端: alpha 档位切换与收敛检测

```

1 typedef enum { ALPHA_COARSE, ALPHA_MEDIUM, ALPHA_FINE } alpha_level_t;
2
3 #define CONVERGE_WINDOW 512 // 收敛检测窗口
4
5 typedef struct {
6     alpha_level_t level;
7     uint32_t up_count; // 估计值增大的次数
8     uint32_t down_count; // 估计值减小的次数
9     int32_t last_est;
10    uint16_t sample_cnt;
11 } converge_detector_t;
12

```

```

13 // 每个通道每个校准类型一个检测器
14 converge_detector_t offset_det[M_CHANNELS];
15
16 alpha_level_t check_convergence(converge_detector_t *det,
17                                 int32_t current_est)
18 {
19     if (current_est > det->last_est)
20         det->up_count++;
21     else if (current_est < det->last_est)
22         det->down_count++;
23
24     det->last_est = current_est;
25     det->sample_cnt++;
26
27     if (det->sample_cnt >= CONVERGE_WINDOW) {
28         // 当增大和减小次数接近相等时，认为收敛
29         int32_t diff = abs((int32_t)det->up_count
30                          - (int32_t)det->down_count);
31
32         if (diff < (CONVERGE_WINDOW / 32)) { // 3%以内
33             switch (det->level) {
34                 case ALPHA_COARSE:
35                     det->level = ALPHA_MEDIUM; break;
36                 case ALPHA_MEDIUM:
37                     det->level = ALPHA_FINE; break;
38                 case ALPHA_FINE:
39                     break; // 已是最精细档位
40             }
41         }
42
43         // 重置计数器
44         det->up_count = 0;
45         det->down_count = 0;
46         det->sample_cnt = 0;
47     }
48
49     return det->level;
50 }

```

1.9.8 SoC 与纯 ASIC 方案对比

1.9.9 关键结论与建议

1. 纯 C 语言方案不可行: 对于 64 GSa/s TIADC, 15.6 ps 的采样间隔远小于单条 C 指令的执行时间。纯软件方案仅对采样率 $\ll 1$ GSa/s 的系统可行。

表 16: ASIC vs SoC 方案全面对比

维度	ASIC (原方案)	SoC (本文方案)
实现方式	Verilog 硬件描述	PL 端 Verilog/HLS + PS 端 C 语言
并行度	全并行（每通道独立运算单元）	PL 端全并行，PS 端串行
位宽控制	硬件固定位宽（29 位/9 位）	PL 同 ASIC, PS 可用更宽位宽
α 档位切换	硬件状态机	C 语言软件灵活控制
收敛检测	逻辑门计数比较	C 语言灵活统计
FFT 性能监测	不支持（需外部仪器）	软件 FFT，在线监测
温度补偿	需额外硬件	软件查表/插值
开发周期	长（流片 ~ 数月）	短（FPGA 可快速迭代）
修改灵活性	低（改版需重新流片）	高（软件更新即可）
单位成本（量产）	低	较高
功耗	低（定制优化）	较高（FPGA + ARM）

2. 推荐 PL + PS 混合架构:

- PL (FPGA): 用 Verilog 或 HLS 实现 Offset/Gain/Timing Skew 的实时数据通路。这部分与原始 ASIC 方案基本一致，可直接复用 Verilog 代码
- PS (ARM): 用 C 语言实现 α 档位切换、收敛检测、性能监测、温度补偿等控制逻辑

3. C 语言的独特价值:

- 灵活参数调整: 不需要重新综合/流片即可修改校准参数
- 在线 FFT 分析: 可在 PS 端周期性地对校准后数据进行 FFT, 实时监测 SNR/SFDR/ENOB
- 自适应策略: 可实现比硬件状态机更复杂的收敛策略（如卡尔曼滤波、机器学习方法）
- 远程调试与固件升级: 通过 OTA 更新校准算法

4. 对于低采样率场景 ($\leq 500 \text{ MSa/s}$): 可考虑在 Cortex-M4/M7 级别 MCU 上用纯 C 语言 + CMSIS-DSP 库实现校准。此时每采样有 $\geq 2\mu\text{s}$ 的处理时间，足够完成定点运算。

5. 推荐的开发流程:

- (a) MATLAB 行为级仿真验证算法（已完成，见前文）
- (b) Vivado HLS / Vitis HLS 将 C 代码综合为 FPGA IP 核
- (c) 在 Zynq / Kria / Versal 平台上进行硬件在环验证
- (d) PS 端开发 C 控制逻辑与性能监测
- (e) 迭代优化至满足目标性能指标

1.10 总结与展望

1.10.1 项目整体架构回顾

本报告的 TIADC 校准系统通过**三步递进式校准**，系统地解决了时间交织 ADC 的三大失配问题。从 PDF 文档中的数学推导和 Verilog 硬件设计，到 MATLAB 代码中的行为级仿真验证，再到 FFT 频谱分析，形成了一个完整的理论 → 仿真 → 验证闭环。

1.10.2 关键技术总结

表 17: 三种校准技术总结

校准类型	核心算法	ASIC 实现方式	SoC 实现方式
Offset	指数平均 (IIR 滤波)	加法器 + 移位寄存器	PL 定点乘加 + PS 档位控制
Gain	LMS 迭代 + 功率比较	乘法器 + 累加器	PL LMS 引擎 + PS 参考功率计算
Timing Skew	自相关 + LMS	乘法器 + VDL 控制	PL 互相关 + PS 误差方向判读

1.10.3 改进方向

- 多档位 α 自适应切换**: 当前代码多使用单档位，SoC PS 端可灵活实现完整的 8 档自动切换逻辑
- 后台校准**: 当前实现为前台校准（需中断正常数据流），实际系统可能需要后台校准（不中断）。SoC 的 PS 端可以在数据流旁路中非侵入式地分析校准质量
- 温度补偿**: 温度变化会引起失配漂移，PS 端可维护温度-参数查找表，实现动态补偿
- 盲校准**: 无需已知输入信号的校准技术，适用于实际工作场景。C 语言的灵活性使盲校准算法更易实现
- 机器学习增强**: 在 PS 端可运行轻量级 ML 模型（如微型 RNN），预测失配参数漂移趋势，实现预测性校准

1.10.4 结语

64 通道 64 GSa/s 的 TIADC 系统代表了当前高速数据转换器的前沿技术方向。通过 SoC (FPGA+ARM) 混合架构，可以结合 ASIC 的实时处理能力和 C 语言的灵活性，构建比纯 ASIC 方案更易迭代、更智能的校准系统。PL 端负责确定性的实时运算，PS 端负责策略性的控制与优化——这一分工最大限度地发挥了两种计算范式的优势。